Question Bank Verification — Drills & Diagnostic

What problem this solves

Every question a student sees — a drill MCQ, a diagnostic Reading passage, a Writing prompt — starts as content someone wrote, not something the database guarantees is correct. Two kinds of things go wrong with hand-authored content at scale:

1. **Structural mistakes** — a malformed options list, a duplicate question, a row filed under the wrong skill, a passage that doesn't match its own question set.

2. **Content mistakes** — the stored "correct answer" is actually wrong, a question is ambiguous or has two defensible answers, or a question is so easy it fails to tell you anything about the student who answers it.

Structural mistakes are cheap to catch by machine. Content mistakes need judgment — traditionally a human proofreader. This tooling automates both passes, so a batch of new questions is checked before it ever reaches a real student, not after someone notices a wrong answer key in production.

There are two independent pipelines — **Drills** (matured first, has been running against real production batches) and **Diagnostic** (built afterward, adapted to a different underlying data shape) — but they share the same two-layer design.

The two-layer design

Layer 1 — Structural Verifier

Reads a staging CSV and checks its shape: nothing here calls an AI, nothing touches a database, and nothing is ever modified. It answers one question: *is this file even well-formed enough to be worth grading?*

Checks include:

- Every required column present, every enum value valid
- Multiple-choice options are well-formed JSON with exactly four non-empty, non-duplicate answers
- The stored "correct answer" actually points at one of the options
- No duplicate questions within a batch
- Cross-checks specific to the content type — e.g. a Reading passage repeated across every question that belongs to it must be *identical* every time; a batch's row count must match what was expected

Output: a colored Excel report (green/amber/red) plus a clean pass/fail exit code, so a batch can be gated automatically.

Layer 2 — AI Content Judge

Only runs on a batch that already passed Layer 1 — a structurally broken row produces meaningless AI judgements. This layer asks whether the content is actually *good*.

For a multiple-choice or True/False/Not Given question, it doesn't just ask the AI "is this right?" — asking a model to grade its own hint is unreliable. Instead:

1. **Blind solve** — the model answers the question fresh, without ever being shown the stored answer.

2. **Compare** — if the model's independent answer matches the stored one, done — that's strong evidence the answer key is correct.

3. **Adjudicate** — only when they *disagree* does a second, more expensive pass run: a referee model is shown both candidate answers and decides which one is actually right, or whether the question itself is broken (ambiguous, or has no correct answer at all).

This "only escalate on disagreement" structure is what keeps the AI cost proportional to genuine uncertainty, rather than paying for a full adjudication on every single row regardless of whether there's anything to adjudicate.

Every judgement is cached by a hash of the question's exact content — fixing one bad row in a 200-row batch re-checks that one row, not all 200 again.

Example: catching a wrong answer key

Suppose a Reading batch has this row:

> **Passage:** *"...Sugar was added to make it sweeter, and it quickly became popular across the continent..."*

> **Question:** "What change made chocolate popular among Europeans?"

> **Options:** A) It was mixed with milk B) Sugar was added to it C) It was made into solid bars D) The price was reduced by half

> **Stored answer:** C

Layer 1 sees nothing wrong — four valid options, a stored answer that's a real key, no duplicates. It passes clean.

Layer 2's blind solve reads the passage and answers **B** independently, with high confidence, because the passage says so explicitly. That disagrees with the stored answer (C), so the adjudicator pass runs: shown both candidates, it agrees the passage clearly supports B, not C, and returns verdict `BLIND_CORRECT`. The report flags this row `ANSWER_WRONG` in red, with the reasoning attached — caught before a single student ever saw it, and without a human having to manually recheck all 200 rows one at a time.

Why this matters specifically for the diagnostic

The diagnostic exists to accurately place a student's true skill level. Two failure modes are worse here than in drills:

- **A wrong answer key** doesn't just cost one wrong drill question — it can misjudge a student's actual band score.
- **A too-easy question** silently defeats the diagnostic's entire purpose. Every AI judgement in the diagnostic pipeline explicitly checks difficulty as a pass/fail criterion, not just correctness — a technically-correct-but-trivial question is flagged the same way a wrong answer is.

The diagnostic pipeline also has to handle content drills never had to: Reading passages that must stay word-for-word identical across a whole question set, and Listening content where the "correct" answer depends on an audio file rather than any text in the row at all. For Listening, the author's submitted transcript is treated as ground truth (since it's literally the script the audio was made from), and the real audio file is separately cross-checked against that transcript using the same AI transcription capability already used elsewhere in the app — catching the one failure mode pure text-checking can't: a recording that doesn't actually match its own script.

Turning this into a microservice

Today, both pipelines are CLI tools an engineer runs by hand against a local file. The path to making this a real service other parts of TestCrack (an admin panel, a content-management flow, a CI pipeline) can call directly:

1. Wrap the existing logic behind an HTTP API — no rewrite needed

The actual checking logic (`verify.ts`, `judge.ts` in both pipelines) is already separated from the CLI's argument-parsing and console-printing concerns. A thin Express layer can call the same functions directly:

```
POST /api/verification/drills/layer1      { csvContent } -> { outcome, findings, reportUrl }
POST /api/verification/drills/layer2      { csvContent } -> { outcome, judgements, reportUrl }
POST /api/verification/diagnostic/layer1  { csvContent } -> { outcome, findings, reportUrl }
POST /api/verification/diagnostic/layer2  { csvContent, audioFiles[] } -> { outcome, judgements,
reportUrl }
```

The CLI keeps working for local/manual use; the API becomes a second entry point into the same core logic.

2. Make Layer 2 asynchronous

AI judging a real batch takes real time (network calls, one per row, plus adjudication passes). A synchronous HTTP request isn't the right shape for that. The natural next step:

```
POST /api/verification/{pipeline}/layer2/jobs     -> { jobId }   (kicks off, returns immediately)
GET  /api/verification/{pipeline}/jobs/{jobId}     -> { status: "running" | "done", progress, res
ults }
```

Backed by a queue (the app doesn't have one yet, so this would be new infrastructure — something like a simple job table + polling, or a proper queue like BullMQ if volume grows) so a batch of 100 questions doesn't hold an HTTP connection open for minutes.

3. Move the report/cache storage off local disk

Right now, `.xlsx` reports and the judgement cache live on the filesystem of whoever runs the CLI. A microservice needs these centralized — object storage (e.g. a Supabase Storage bucket, matching the pattern already used elsewhere in the app) for reports, and the cache moved from local JSON files to a proper table (`verification_cache`, keyed by the same content hash already computed today) so cache hits work across every machine calling the service, not just one engineer's laptop.

4. Add the missing piece: real import, gated on a clean verdict

Neither pipeline currently writes to the database — that's deliberate today (no `--confirm` flag exists on purpose, matching the drills tooling's own safety philosophy). A microservice version would add a final gated step:

```
POST /api/verification/diagnostic/import     { jobId }  -> only allowed if that job's Layer 1 AND
 Layer 2 outcome was clean
```

This is the point where an admin-panel "Upload new batch → Verify → Review flagged rows → Approve → Goes live" flow becomes possible end-to-end, instead of an engineer manually running two CLI commands and then hand-writing an import script.

5. Where this fits alongside the rest of TestCrack's services

The verification logic doesn't need its own database or its own deploy target initially — it can live as a set of routes inside the existing backend, reusing the same Prisma connection, the same Gemini API key management, and the same auth middleware that gates other admin-only endpoints. Splitting it into a genuinely separate microservice (its own process, its own scaling) only becomes worth doing if judging volume grows enough that AI-judging load needs to scale independently of the main API — not a v1 concern.